Benchmarking Vendor-Guided Optimizations on Edge NPUs: From NPU Only to End-to-End Performance

Youngjin Jo[1,2][0009−0000−8772−5837], Jihwan Kim[1][0009−0000−3446−4805], Minseok Choi[1][0009−0003−6346−4144], Kwanghoon Choi[1][0000−0003−3519−3650], Jusung Kang[2][0000−0002−7908−9452], and Jaein Kim[2][0009−0006−0739−9052]

[1] Chonnam National University, Gwangju, 61186, Korea
{jo.youngjin,213652,213837,kwanghoon.choi}@jnu.ac.kr
[2] Electronics and Telecommunications Research Institute, Gwangju, 61012, Korea
{k92492,jaein}@etri.re.kr

Abstract. Edge NPU performance depends not only on hardware capabilities, but also on model transformation and the overall deployment pipeline. We propose a benchmarking methodology that separates these effects by comparing three configurations: Baseline, Vendor Guided NPU Only (VG-NPU), and Vendor Guided End-to-End (VG-E2E), where VG-NPU preserves common model I/O and host-side processing while VG-E2E follows the vendor-native deployment pipeline. We apply this methodology to four edge NPUs and three vision workloads. VG-NPU improves both accuracy and runtime-call latency in 10 of 12 NPU–workload combinations, whereas VG-E2E reduces System-level latency in all combinations but improves accuracy in only four. Moreover, NPU-level performance rankings do not always match End-to-End rankings, and VG-E2E provides an additional 10.4–70.7% reduction in System-level latency over VG-NPU. These results demonstrate the importance of evaluating vendor-guided optimization separately at the controlled model-transformation and full deployment levels.

Keywords: Edge NPU · Benchmarking · Vendor-Guided Optimization · Inference Performance · End-to-End Evaluation

1 Introduction

As AI deployment expands, efficient inference has become increasingly important, particularly in mobile and edge environments with tight power and resource constraints. Neural Processing Units (NPUs) address this demand by accelerating tensor operations commonly used in neural networks, often providing higher efficiency than general-purpose CPUs and GPUs for supported workloads [13]. In practice, however, NPU inference performance depends not only on hardware compute capability, but also on the surrounding software stack, including operator support, compiler optimizations, quantization, runtime execution, and input/output processing.

Evaluating NPU-based inference systems requires considering both accuracy and latency. Accuracy reflects how well the predictive quality of the original model is preserved during deployment to an NPU, while latency measures the execution time required to produce the final prediction from an input. In particular, latency can differ substantially depending on whether it measures only NPU inference or end-to-end execution including preprocessing and postprocessing. This study focuses on how accuracy and latency change across the deployment pipelines of different NPUs.

Comparing different NPUs fairly is challenging. NPUs differ not only in hardware architecture and supported operators, but also in their compilers, quantization policies, precision settings, runtime interfaces, and preprocessing/postprocessing pipelines. Consequently, even when starting from the same source model, applying vendor-recommended model transformations and deployment recipes can change not only the final executable model, but also which operations are executed on the NPU and how the overall deployment pipeline is constructed. As a result, comparing only the final end-to-end (E2E) performance reported for each vendor makes it difficult to determine whether an observed performance difference arises from model transformations performed for NPU execution or from the broader deployment pipeline, including preprocessing, postprocessing, and runtime overheads.

Prior work has proposed benchmarking methodologies and evaluation metrics for fair comparison of ML inference systems [13, 12], and has analyzed how preprocessing, postprocessing, framework, and runtime overheads beyond model inference affect end-to-end performance [11, 4]. Other studies have shown that the execution performance of the same model can vary depending on quantization, compilers, deployment tools, and hardware-specific optimizations [7, 1]. However, these studies have primarily focused on cross-platform comparison, system-level overheads, or the effects of individual optimization techniques. They have not explicitly separated the effect of vendor-guided optimization into two levels: model transformation under fixed model inputs, outputs, and host-side processing, and actual vendor-native end-to-end deployment. By distinguishing these two levels, this study separately examines how model transformation and full deployment affect accuracy and latency.

To investigate this research gap, we define three experimental conditions: Baseline, Vendor-Guided NPU-Only (VG-NPU), and Vendor-Guided End-to-End (VG-E2E). Baseline serves as a reference configuration that permits only the transformations required for NPU execution while limiting optional vendor-specific optimizations. VG-NPU applies the vendor-guided model transformation recipe while preserving the same model inputs and outputs and the same host-side processing as Baseline. VG-E2E applies the full vendor-recommended deployment configuration, including native input/output formats, preprocessing, runtime execution, and postprocessing. With this design, VG-NPU captures the controlled effect of model transformation, whereas VG-E2E measures the performance of the final vendor-guided deployment.

Following the proposed evaluation methodology, we analyze three research questions using four NPUs and three vision workloads: ResNet50, YOLOv5m, and YOLOv8s-pose. First, under VG-NPU, both latency and accuracy improve in 10 of the 12 NPU–workload combinations. In contrast, under VG-E2E, System-level latency decreases in all combinations, while accuracy decreases in eight. This result shows that vendor-guided optimization can improve both metrics at the controlled model-transformation level, but that lower latency in the final deployment does not necessarily imply preserved accuracy.

Second, when comparing three NPUs on the same host, the relative performance ranking among NPUs remains unchanged for YOLOv5m and YOLOv8s-pose, but changes for ResNet50. This indicates that NPU-only performance does not, in general, reliably predict actual end-to-end performance, and that the relationship is workload-dependent.

Third, VG-E2E achieves lower System-level latency than VG-NPU in all combinations. This suggests that deployment-level processing can provide additional performance gains, while also indicating that such gains should not be attributed to any single optimization component. Overall, these results show that Edge NPU performance should be evaluated separately at the NPU-level optimization and final deployment levels, rather than being treated as a single performance metric.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the three experimental configurations, workloads, NPU platforms, and measurement methodology. Section 4 presents the main experimental results and discusses the limitations of the study. Finally, Section 5 concludes the paper and outlines directions for future work.

2 Related Work

Benchmarking methodology and evaluation metrics: MLPerf Inference [12] standardizes workloads, accuracy targets, execution scenarios, and measurement rules to enable fair and reproducible comparisons of ML inference systems across diverse hardware and software platforms. MLPerf Tiny [3] extends this benchmarking approach to ultra-low-power TinyML environments by jointly measuring latency, energy, and accuracy. Its modular benchmark design also enables the contributions of different layers of the deployment stack, including models, frameworks, and hardware, to be analyzed separately. These efforts demonstrate that, even across substantially different application domains, fair evaluation of accuracy–latency–energy trade-offs requires considering multiple performance metrics and separating the contributions of individual system layers.

Sze et al. [13] similarly emphasize that DNN inference systems should be evaluated using multiple metrics, including accuracy, latency/throughput, energy/power, and hardware cost, and that performance should be measured under representative workloads and runtime conditions. Coleman et al. [5] proposed DAWNBench, highlighting the need for end-to-end system evaluation that cap-

tures the combined effects of model architecture, optimization procedures, software frameworks, and hardware.

Following these evaluation principles, this study distinguishes controlled NPU-only execution from native end-to-end deployment to isolate and analyze the effects of vendor-guided optimization.

NPU/system-level and E2E performance analysis: Millar et al. [11] compared microcontroller-scale μNPUs under common workloads and measurement conditions, decomposing execution time into initialization, memory I/O, inference, and post-processing to identify end-to-end performance bottlenecks.

Buch et al. [4] characterized the overheads beyond model inference in mobile AI—including data capture, pre/post-processing, framework, and offloading overheads—as the "AI tax," and analyzed their impact on application-level end-to-end performance.

Szegedi et al. [14] separately measured preprocessing, inference, and post-processing for classification and detection models on edge GPUs, and compared performance across frameworks, implementation languages, and precision settings, demonstrating the need to consider both model inference time and total processing time.

Kang et al. [9] showed that, in accelerator-based visual analytics systems, preprocessing operations such as image decoding and resizing can become throughput bottlenecks that are more significant than model inference itself.

Ham et al. [6], through their work on NNStreamer, demonstrated that execution configuration can substantially affect the performance of on-device AI pipelines composed of multiple processing stages. By connecting neural network inference and data processing through GStreamer and comparing sequential execution with stream-pipelined execution, they showed that the organization of data flow and parallel execution affects overall latency, throughput, and resource utilization.

While these studies demonstrate the importance of system-level overheads beyond NPU inference, our work distinguishes vendor optimization on commercial Edge NPUs into NPU-only model transformation and full deployment, and compares the performance impact at each level.

Optimization/toolchain effects: EdgeMark [7] proposed a benchmarking system that automates model generation, optimization, conversion, and deployment for embedded AI tools, enabling multiple toolchains to be compared under consistent conditions.

Ahn et al. [1] compared INT8/FP16 and static/dynamic quantization across different frameworks and edge hardware, showing that the effectiveness of the same model optimization can vary depending on software frameworks and hardware support. Baciu et al. [2] similarly compared porting frameworks and quantization configurations on real MCUs, showing that model quantization can reduce model size and classification time at the cost of reduced accuracy.

Ulker et al. [15] showed that inference performance depends on both the computational structure of a model and framework-specific operator implemen-

tations, while MLonMCU [10] demonstrated that inference performance and the gains from auto-tuning can vary with TVM schedules and data layouts.

In contrast to these studies, our work does not focus on comparing individual quantization techniques or deployment tools. Instead, we analyze how complete vendor-recommended optimization recipes affect controlled NPU-only performance and actual end-to-end deployment performance, considering both accuracy and latency.

3 Benchmark Design

To distinguish the effects of vendor-provided NPU optimizations on accuracy and latency between model transformation and final deployment, we define three benchmark configurations: Baseline, Vendor-Guided NPU-Only (VG-NPU), and Vendor-Guided End-to-End (VG-E2E). By comparing Baseline and VG-NPU, we examine the changes introduced by vendor-guided model transformation, while VG-E2E is used to evaluate the final performance of the full vendor-recommended deployment configuration. In Fig. 1, we illustrate the three configurations, the execution paths, and the corresponding latency measurement methodology.

3.1 Benchmark Framework

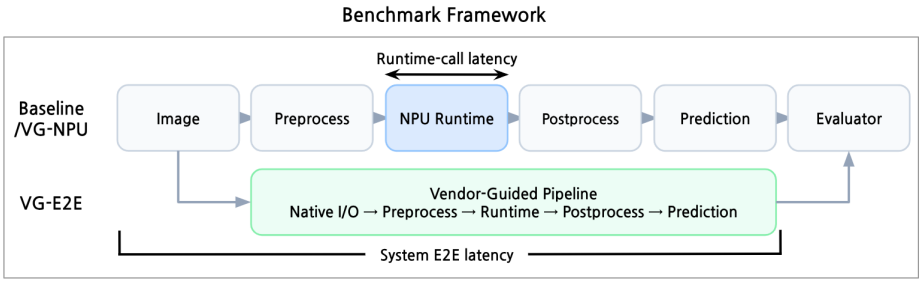


Fig. 1. Execution paths and latency measurement intervals of the benchmark framework. Baseline and VG-NPU use the same execution path, whereas VG-E2E follows the vendor-guided deployment pipeline. Runtime-call latency measures the elapsed time of the runtime API invocation, which includes NPU execution and any overhead incurred within the runtime call, while System-level latency measures the total execution time from image input to final prediction.

Fig. 1 illustrates the execution paths of the three benchmark configurations. Baseline and VG-NPU use the same benchmark framework and generate the final prediction through identical preprocessing, NPU runtime invocation, and post-processing steps. The two configurations are also designed to use the same model

inputs and outputs. The primary difference between Baseline and VG-NPU is the model transformation configuration used to generate the NPU-executable model. In contrast, VG-E2E uses the vendor-recommended deployment pipeline, including native input/output formats, preprocessing, runtime, and postprocessing. Accuracy is evaluated using the same evaluator for all configurations.

Runtime-call latency measures the runtime API invocation, while System-level latency measures the time from image loading to final prediction. Since Baseline and VG-NPU use the same runtime interface, their model-transformation effects are compared using Runtime-call latency. Overall deployment performance, including VG-E2E, is compared using System-level latency.

All experiments use synchronous single-stream execution with batch size 1 and the same 5,000-image evaluation set. After 10 warm-up runs, the full set is executed four times. We take the median latency per image across the four runs and report the mean over all 5,000 images.

3.2 Benchmark Configurations

Table 1. Benchmark Configurations

Category	**Baseline**	**VG-E2E**	**VG-NPU**
Source model / weights	Same	Same	Same
Calibration / evaluation	1,024 / 5,000 images	Same	Same
Batch / execution	1 / Sync. / Single-stream	Same	Same
Compiler input	Source-model subgraph	Source model	Source model subgraph
Quantization	INT 8-bit PTQ	Vendor options	Vendor options
Compiler optimization	Minimal settings	Vendor options	Vendor options
Execution path	Benchmark pipeline	vendor pipeline	Benchmark pipeline
Latency	Runtime-call / System-level	System-level	Runtime-call / System-level
Evaluation focus	Trasnformation baseline	Final deployment performance	Model-transformation effect

The Baseline configuration serves as the reference point for evaluating the effects of the two vendor-guided optimization configurations.

– Model conversion and INT8 post-training quantization are required to make the model executable on each NPU, and these are also applied in the Baseline configuration, since the original FP32 model cannot be executed directly on the NPU without these transformations.

– In contrast, vendor-specific optimizations that are not required for model execution are disabled in the Baseline configuration. These include fine-tuning, mixed precision, weight compression, enhanced quantization, graph-level performance optimization, and resource-allocation tuning. These optimizations are enabled only in the vendor-guided configurations, allowing their effects on performance and accuracy to be measured relative to the Baseline.

VG-E2E evaluates the deployment performance of the vendor-native execution path, including preprocessing, NPU inference, and postprocessing, using the deployment modules and recommended model-transformation settings provided by each vendor.

– Model transformation uses the vendor-recommended model graph, quantization, and compiler optimization settings, while execution uses native input/output formats and vendor-specific preprocessing, runtime, and postprocessing.
– Therefore, changes observed under VG-E2E are interpreted as the effect of the overall vendor-guided deployment rather than that of any single optimization technique.

To isolate the effects of NPU inference and model transformation from the overall E2E performance changes, we additionally define the VG-NPU configuration.

– VG-NPU follows the same execution setup as Baseline, but applies the vendor-recommended model-transformation recipe, including quantization and compiler optimizations.
– Therefore, the performance difference between Baseline and VG-NPU captures the effect of vendor-recommended model transformation on NPU inference.

3.3 Workloads and Datasets

To account for the possibility that NPU optimization effects vary with workload computation and output-processing characteristics, we select ResNet50, YOLOv5m, and YOLOv8s-pose to represent image classification, object detection, and pose estimation, respectively.

ResNet50 represents image classification, YOLOv5m object detection, and YOLOv8s-pose human pose estimation. Accuracy is evaluated based on the final semantic predictions. For Baseline and VG-NPU, identical model outputs and a common postprocessing pipeline are used so that differences in NPU-specific output tensor layouts do not affect the comparison. For the YOLO workloads, decoding, confidence filtering, and NMS are performed using the same common postprocessor across all NPUs. Detailed output tensor configurations and the threshold settings used for accuracy and latency measurements are provided in the extended technical report [8].

Table 2. Evaluation workloads, datasets, and accuracy metrics.

Model / Task	**Input**	**Dataset / Format**	**Cal. / Eval.**	**Metric**	**FP32 Score**
ResNet50 / Classification	RGB 224×224	ImageNet-1K / JPEG + class label	train 1,024 / val 5,000	Top-1 accuracy	79.06
YOLOv5m / Detection	RGB 640×640	COCO 2017 / JPEG + bbox annotation	train2017 1,024 / val2017 5,000	bbox AP@[.50:.95]	45.16
YOLOv8s-pose / Pose estimation	RGB 640×640	COCO 2017 / JPEG + keypoint annotation	train2017 1,024 / val2017 5,000	keypoint AP@[.50:.95]	59.39

All accuracy metrics are reported on a 0–100 scale. ResNet50 uses Top-1 accuracy (%), while YOLOv5m and YOLOv8s-pose use COCO bbox AP and keypoint AP, respectively, multiplied by 100. Bbox AP is averaged over IoU thresholds from 0.50 to 0.95 in increments of 0.05, and keypoint AP is averaged over the corresponding OKS thresholds.

For accuracy evaluation, we use ImageNet-1K for ResNet50 and COCO val2017 for YOLOv5m and YOLOv8s-pose. Each workload is evaluated on 5,000 images, while calibration uses a fixed set of 1,024 images from the corresponding training dataset. For each workload, the same calibration and evaluation images, in the same order, are used across all NPUs and benchmark configurations to control for dataset-related variation.

3.4 Vendor-Specific Deployment Configuration

For each NPU, we apply the optimization and deployment configurations recommended by the corresponding vendor, including quantization, precision, graph transformation, preprocessing/postprocessing, and runtime pipeline.

- Specifically, Hailo uses optimization/compression-level tuning, mixed precision, and quantization fine-tuning, together with a GStreamer-based execution path and model-integrated normalization or NMS.
- DeepX applies compiler optimization levels and enhanced quantization, and uses a DXSTREAM-based pipeline.
- Mobilint applies compiler optimization, percentile-based quantization, and Hadamard transformation, using a Model Zoo- and `qbruntime`-based execution setup.

In VG-NPU, we primarily apply quantization and compiler-level model transformations from these vendor settings, whereas VG-E2E additionally includes vendor-native preprocessing, model-integrated processing, postprocessing, and runtime pipelines. Detailed model-specific options and exceptional settings are summarized in the extended technical report [8].

3.5 NPU Hardware and Software Environments

We selected four commercially available Edge NPUs from multiple vendors that provide dedicated compiler, runtime, and deployment stacks required to evaluate both the VG-NPU and VG-E2E configurations. Table 3 summarizes the hardware and software environments used for each platform.

Table 3. Evaluation NPU Platforms and Software Stacks

Target	**INT8 TOPS**	**Host / OS**	**CPU Freq.**	**Compiler**	**Runtime**
Hailo-8	26	Jetson Orin Nano / Ubuntu 22.04, JetPack 6	1.728 GHz	DFC 3.34	HailoRT 4.24
Hailo-10H	20	Jetson Orin Nano / Ubuntu 22.04, JetPack 6	1.728 GHz	DFC 5.3	HailoRT 5.3
DeepX DX-M1	25	Jetson Orin Nano / Ubuntu 22.04, JetPack 6	1.728 GHz	DX-COM 2.3-rc.5	DX-RT 3.4
Mobilint Regulus	10	Regulus SOM / Vendor Yocto Linux	1.2 GHz	QBCompiler 1.1.2	qbruntime 1.2

All NPUs are evaluated using the hardware and software stacks provided by their respective vendors. Hailo-8, Hailo-10H, and DeepX DX-M1 are measured on the same Jetson Orin Nano host, while Mobilint Regulus is evaluated on the vendor-provided Regulus SOM. To reduce run-to-run variation caused by host-side frequency scaling, the CPU frequency is fixed at 1.728 GHz on the Jetson Orin Nano and 1.2 GHz on the Regulus System on Module (SOM), respectively.

4 Evaluation Results and Findings

In this evaluation, we examine how vendor-guided optimization affects latency and accuracy, and how NPU-only performance translates into end-to-end performance. Table 4 summarizes the accuracy measured under Baseline, VG-NPU, and VG-E2E, while Table 5 reports the System-level latency for the three configurations. Table 6 compares the Runtime-call latency of Baseline and VG-NPU.

4.1 RQ1: Do vendor-guided optimizations improve both latency and accuracy?

Comparing Baseline and VG-NPU, accuracy improves in 10 of the 12 NPU–workload combinations, while Runtime-call latency decreases in 11. Both metrics

Table 4. Accuracy Results on a 0–100 Scale

NPU	Model	Baseline	VG-NPU	VG-E2E
Hailo-8	ResNet50	78.38	78.88	78.08
	YOLOv5m	41.68	43.33	39.21
	YOLOv8s-pose	49.67	51.68	48.29
Hailo-10H	ResNet50	78.32	78.92	77.82
	YOLOv5m	42.39	43.46	42.80
	YOLOv8s-pose	50.58	52.49	47.84
DeepX DX-M1	ResNet50	78.86	78.66	78.94
	YOLOv5m	44.75	44.82	43.69
	YOLOv8s-pose	58.47	58.90	58.54
Mobilint Regulus	ResNet50	78.74	78.96	77.42
	YOLOv5m	43.90	42.89	43.38
	YOLOv8s-pose	56.25	56.57	56.66

Table 5. System-level Latency (ms).

NPU	Model	Baseline	VG-NPU	VG-E2E
Hailo-8	ResNet50	11.100	9.786	8.766
	YOLOv5m	59.308	43.106	28.101
	YOLOv8s-pose	43.938	38.228	22.167
Hailo-10H	ResNet50	16.145	15.082	9.712
	YOLOv5m	124.871	114.839	33.697
	YOLOv8s-pose	92.837	89.904	32.683
DeepX DX-M1	ResNet50	17.739	17.237	7.243
	YOLOv5m	122.177	121.915	53.768
	YOLOv8s-pose	106.322	105.563	55.149
Mobilint Regulus	ResNet50	34.217	34.233	28.187
	YOLOv5m	122.216	122.332	96.966
	YOLOv8s-pose	193.581	192.910	86.611

improve simultaneously in 10 combinations, indicating that vendor-guided model transformation can generally reduce NPU inference latency while preserving or slightly improving accuracy under fixed model inputs, outputs, and host-side processing. However, accuracy decreases for YOLOv5m on Mobilint Regulus, and several NPU–workload combinations show only marginal changes in Runtime-call latency, indicating that the benefits are not uniform across all cases.

In contrast, under VG-E2E, System-level latency decreases relative to Baseline in all 12 NPU–workload combinations, whereas accuracy improves in only four and decreases in eight. Thus, vendor-guided deployment consistently improves latency, but does not necessarily improve accuracy. In other words, full-deployment optimization can achieve lower latency at the cost of accuracy degradation for some workloads.

Table 6. Runtime-call Latency under the Baseline and Vendor Guided NPU Only Configurations (ms).

NPU	Model	Baseline	VG-NPU
Hailo-8	ResNet50	5.506	4.187
	YOLOv5m	39.640	23.271
	YOLOv8s-pose	18.656	12.891
Hailo-10H	ResNet50	10.512	9.462
	YOLOv5m	105.090	95.024
	YOLOv8s-pose	67.445	65.157
DeepX DX-M1	ResNet50	11.862	11.449
	YOLOv5m	97.082	96.851
	YOLOv8s-pose	83.571	82.892
Mobilint Regulus	ResNet50	5.270	5.269
	YOLOv5m	28.558	28.568
	YOLOv8s-pose	19.409	18.822

4.2 RQ2: Does NPU-only performance predict end-to-end performance?

To examine whether NPU-only performance can predict actual end-to-end performance, we compare the relative performance rankings among NPUs using VG-NPU Runtime-call latency and VG-E2E System-level latency. To minimize the effect of host differences, we focus on Hailo-8, Hailo-10H, and DeepX DX-M1, all measured on the same Jetson Orin Nano host. Mobilint Regulus is excluded from this comparison because it was measured on a separate vendor-provided Regulus SOM.

For ResNet50, the performance ranking differs between NPU-only and end-to-end measurements. Under VG-NPU, Runtime-call latency follows Hailo-8 < Hailo-10H < DeepX DX-M1, whereas under VG-E2E, System-level latency follows DeepX DX-M1 < Hailo-8 < Hailo-10H. Notably, DeepX DX-M1, which is the slowest in the NPU-only measurement, achieves the lowest E2E latency under VG-E2E. This shows that NPU inference performance alone does not always predict overall deployment performance.

In contrast, for YOLOv5m and YOLOv8s-pose, the relative ranking among NPUs remains unchanged across the two measurements. For both workloads, the ranking is Hailo-8 < Hailo-10H < DeepX DX-M1 for both VG-NPU Runtime-call latency and VG-E2E System-level latency. These results show that the relationship between NPU-only and end-to-end performance is workload-dependent, and that overall system performance can be influenced not only by NPU inference but also by preprocessing, postprocessing, and vendor-specific execution pipelines.

4.3 RQ3: How much additional benefit comes from vendor-guided deployment?

To examine whether vendor-guided deployment provides additional performance gains beyond NPU-level optimization, we compare the System-level latency of

VG-NPU and VG-E2E. VG-E2E achieves lower latency than VG-NPU in all 12 NPU–workload combinations, with reductions ranging from 10.4% to 70.7%. Particularly large additional improvements are observed for YOLOv5m and YOLOv8s-pose on Hailo-10H, all three workloads on DeepX DX-M1, and YOLOv8s-pose on Mobilint Regulus.

VG-NPU and VG-E2E apply the same vendor-guided model transformation, but VG-E2E additionally uses vendor-specific preprocessing, postprocessing, and runtime pipelines. Since VG-E2E achieves lower E2E latency than VG-NPU in all combinations, the additional latency gains can be attributed to system-level processing beyond NPU-internal model transformation. This suggests that deployment-level optimization plays an important role in further reducing overall latency. However, because multiple components are changed simultaneously, the observed gains cannot be isolated as the effect of any single preprocessing or postprocessing technique.

4.4 Limitations

This study is limited to four NPUs, three vision workloads, specific software stacks, and synchronous batch-1 execution, and does not explore all compiler and quantization configurations. In Baseline and VG-NPU, some full-graph optimizations are restricted to preserve identical model inputs, outputs, and host-side processing. In VG-E2E, multiple deployment components change simultaneously, preventing us from isolating the effect of individual optimizations. Because some platforms use different hosts, absolute E2E latency cannot be attributed solely to NPU performance. Power and energy-efficiency evaluation was excluded because the available power telemetry interfaces and measurement coverage differed across platforms, making it difficult to establish a consistent power measurement interval. Consequently, we cannot determine whether the observed latency improvements translate into better power efficiency or lower energy per inference. Thus, our results should be interpreted as the observed effects in vendor-guided optimization within the scope of our experiments, rather than as maximum achievable performance or universal superiority.

5 Conclusion and Future Work

Using four commercial Edge NPUs and three widely used vision workloads spanning image classification, object detection, and pose estimation, our findings show that Edge NPU performance should be interpreted by distinguishing NPU-level model transformation from final deployment, rather than treating inference performance as a single metric. First, vendor-guided model transformation generally reduces NPU inference latency while preserving accuracy under fixed model inputs, outputs, and host-side processing. Second, such NPU-level improvements do not always translate directly into final End-to-End performance; for some workloads, the relative performance ranking among NPUs changes after

full deployment. Third, vendor-native deployment provides additional System-level latency reductions across all evaluated combinations, but these performance gains do not always preserve accuracy. These findings suggest that comparing Edge NPUs requires considering the latency–accuracy trade-off jointly shaped by model transformation, preprocessing, postprocessing, and runtime pipelines, rather than NPU execution alone.

Future work will extend the evaluation to a broader range of NPUs and workloads, isolate the effects of individual optimizations, and incorporate workload-specific accuracy constraints to better understand the sources of these trade-offs.

References

1. Ahn, H., Chen, T., Alnaasan, N., Shafi, A., Abduljabbar, M., Subramoni, H., Panda, D.K.: Performance Characterization of Using Quantization for DNN Inference on Edge Devices. In: 2023 IEEE 7th International Conference on Fog and Edge Computing (ICFEC). pp. 1–6. IEEE Computer Society, Los Alamitos, CA, USA (May 2023). https://doi.org/10.1109/ICFEC57925.2023.00009
2. Baciu, V.E., Stiens, J., {da Silva}, B.: Mlino bench: A comprehensive benchmarking tool for evaluating ml models on edge devices. Journal of Systems Architecture **155**(103262) (Oct 2024). https://doi.org/10.1016/j.sysarc.2024.103262, presented at AEIC2024 Barcelona
3. Banbury, C., Reddi, V.J., Torelli, P., Holleman, J., Jeffries, N., Kiraly, C., Montino, P., Kanter, D., Ahmed, S., Pau, D., Thakker, U., Torrini, A., Warden, P., Cordaro, J., Guglielmo, G.D., Duarte, J., Gibellini, S., Parekh, V., Tran, H., Tran, N., Wenxu, N., Xuesong, X.: Mlperf tiny benchmark. In: Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS 2021), Track on Datasets and Benchmarks. Curran Associates, Inc. (December 2021). https://doi.org/10.48550/arXiv.2106.07597, 35th Conference on Neural Information Processing Systems (NeurIPS 2021), Track on Datasets and Benchmarks
4. Buch, M., Azad, Z., Joshi, A., Reddi, V.J.: AI tax in mobile socs: End-to-end performance analysis of machine learning in smartphones. In: 2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS). pp. 96–106 (2021). https://doi.org/10.1109/ISPASS51385.2021.00027
5. Coleman, C.A., Narayanan, D., Kang, D., Zhao, T., Zhang, J., Nardi, L., Bailis, P., Olukotun, K., Ré, C., Zaharia, M.: Dawnbench: An end-to-end deep learning benchmark and competition. In: 31st Conference on Neural Information Processing Systems (NIPS 2017) (2017), https://dawn.cs.stanford.edu/sites/g/files/sbiybj26306/files/media/file/nips17-dawnbench.pdf
6. Ham, M., Moon, J., Lim, G., Jung, J., Ahn, H., Song, W., Woo, S., Kapoor, P., Chae, D., Jang, G., Ahn, Y., Lee, J.: Nnstreamer: efficient and agile development of on-device ai systems. In: Proceedings of the 43rd International Conference on Software Engineering: Software Engineering in Practice. p. 198–207. ICSE-SEIP '21, IEEE Press (2021). https://doi.org/10.1109/ICSE-SEIP52600.2021.00029
7. Hasanpour, M.A., Kirkegaard, M., Fafoutis, X.: EdgeMark: An automation and benchmarking system for embedded artificial intelligence tools. Journal of Systems Architecture **167**, 103488 (2025). https://doi.org/https://doi.org/10.1016/j.sysarc.2025.103488

8. Jo, Y., Kim, J., Choi, M., Choi, K., Kang, J., Kim, J.: Benchmarking vendor-guided optimizations on edge NPUs: From NPU-only to end-to-end performance. Extended technical report, Chonnam National University, Gwangju, Korea (2026)
9. Kang, D., Mathur, A., Veeramacheneni, T., Bailis, P., Zaharia, M.: Jointly optimizing preprocessing and inference for dnn-based visual analytics. Proc. VLDB Endow. **14**(2), 87–100 (Oct 2020). https://doi.org/10.14778/3425879.3425881
10. van Kempen, P., Stahl, R., Mueller-Gritschneder, D., Schlichtmann, U.: Mlonmcu: Tinyml benchmarking with fast retargeting. In: Proceedings of the 2023 Workshop on Compilers, Deployment, and Tooling for Edge AI. p. 32–36. CODAI '23, Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3615338.3618128
11. Millar, J., Huang, Y., Sethi, S., Haddadi, H., Madhavapeddy, A.: Benchmarking ultra-low-power µNPUs. In: Proceedings of the 31st Annual International Conference on Mobile Computing and Networking. p. 1060–1074. ACM MOBICOM '25, Association for Computing Machinery, New York, NY, USA (2025). https://doi.org/10.1145/3680207.3765264
12. Reddi, V.J., Cheng, C., Kanter, D., Mattson, P., Schmuelling, G., Wu, C.J., Anderson, B., Breughe, M., Charlebois, M., Chou, W., Chukka, R., Coleman, C., Davis, S., Deng, P., Diamos, G., Duke, J., Fick, D., Gardner, J.S., Hubara, I., Idgunji, S., Jablin, T.B., Jiao, J., John, T.S., Kanwar, P., Lee, D., Liao, J., Lokhmotov, A., Massa, F., Meng, P., Micikevicius, P., Osborne, C., Pekhimenko, G., Rajan, A.T.R., Sequeira, D., Sirasao, A., Sun, F., Tang, H., Thomson, M., Wei, F., Wu, E., Xu, L., Yamada, K., Yu, B., Yuan, G., Zhong, A., Zhang, P., Zhou, Y.: MLPerf inference benchmark. In: Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture. p. 446–459. ISCA '20, IEEE Press (2020). https://doi.org/10.1109/ISCA45697.2020.00045
13. Sze, V., Chen, Y.H., Yang, T.J., Emer, J.S.: Efficient processing of deep neural networks: A tutorial and survey. Proceedings of the IEEE **105**(12), 2295–2329 (2017). https://doi.org/10.1109/JPROC.2017.2761740
14. Szegedi, R., Bukschat, Y., Vetter, M.: A comparative study of neural network performance on the nvidia jetson orin nano. IEEE Access **14**, 107765–107785 (2026). https://doi.org/10.1109/ACCESS.2026.3712902
15. Ulker, B., Stuijk, S., Corporaal, H., Wijnhoven, R.: Reviewing inference performance of state-of-the-art deep learning frameworks. In: Stuijk, S. (ed.) SCOPES '20: Proceedings of the 23th International Workshop on Software and Compilers for Embedded Systems. pp. 48–53. Association for Computing Machinery, Inc., United States (May 2020). https://doi.org/10.1145/3378678.3391882, 23rd International Workshop on Software and Compilers for Embedded Systems (SCOPES 2020) ; Conference date: 25-05-2020 Through 26-05-2020